Group Members Name	Hafiz Umar Khan 20096793
Programme:	Master of Cyber Security
Lecturer Name:	Swati Dongre
Module/Subject Title:	Advanced Programming Techniques
Assignment Title:	Continuous Assessment 1
Github Link	https://github.com/umer2122/ADVANCED_PROGRAMMING_TECHNIQUES_CA1/blob/main/README.md

By submitting this assignment, I am/ we are confirming that:

1. This assignment is all our own work.
2. Any sources used have been referenced.
3. I/we have followed the Generative AI instructions/ scale set out in the Assignment Brief.
4. I/we have read the College rules regarding academic integrity in the QAH Part B Section 3, and the Generative AI Guidelines, and understand that penalties will be applied accordingly if work is found not to be my/our own.
5. I/we understand that all work uploaded is submitted via Ouriginal, whereby a text-matching report will show any similarities with other texts.

Introduction

The assignment displays how practical solutions can be developed in C# and Python and concentrates on data structure, data management, and external integrations. The most important concepts are object-oriented design, communication by use of TCP between a client and a server, storage of data in a database and web scraping. Different types of data, including CSV, HTML, databases, are used to develop effective systems, and external libraries and APIs are incorporated to satisfy the needs of the real-world application.

Task 1: Churros Ordering System (C#)

1. Developed a **menu-driven console application** to simulate a food truck ordering system where the operator can place and deliver orders.
2. Implemented two main classes:
   - **Order.cs**: Contains attributes such as ***order_no***, ***order_details***, ***quantity***, and ***bill***. Includes methods like ***PayBill()*** and ***CollectOrder()***.
   - **Churros.cs**: Stores item name and price, representing different churros options.
3. Applied key **Object-Oriented Programming (OOP)** concepts:
   - Encapsulation using private setters and controlled access
   - Constructors to initialise objects
   - Properties for safe data handling
4. Used a **Queue data structure *(Queue<Order>)*** to manage orders in a **FIFO (First-In-First-Out)** manner, ensuring fair processing of customers.
5. The **Program.cs** file contains the main logic:
   - Displays menu options (place order, deliver order, exit)
   - Handles user input and order creation
   - Adds orders to the queue and removes them upon delivery
6. Implemented a **bill calculation method *(PayBill)*** that computes total cost based on quantity and price.
7. A **unit test** was created to validate the correctness of the bill calculation method.
8. The system demonstrates structured programming, efficient data handling, and real-world application of OOP and data structures.

Task 2: Periodic Table System (C#)

1. Developed a **console-based application** to store and retrieve information about the first 30 elements of the periodic table based on user input.
2. Utilised a **Dictionary data structure *(Dictionary<int, Element>)*** to store elements, where the **atomic number acts as the key** for efficient and direct access.
3. Implemented the main logic in **Program.cs**:
   - Prompts the user to enter an atomic number (1–30)
   - Retrieves and displays corresponding element details
   - Allows repeated queries using a loop until the user exits
4. Stored element details including:
   - **Name** (e.g., Hydrogen)
   - **Class** (e.g., Nonmetal, Noble Gas)
   - **Additional information** about the element
5. Included **input validation** to ensure only valid atomic numbers are accepted, improving program robustness.
6. Demonstrated effective use of **data structures** for fast data retrieval and **control structures** (loops and conditionals) for user interaction.
7. The system provides a simple and efficient way to access structured scientific data, highlighting practical implementation of data storage and retrieval techniques in C#.

Task 3: EasyDrive Client-Server Application (Python)

1. Developed a **client-server application** using Python to simulate a car rental registration system based on the EasyDrive scenario.
2. Implemented two separate files as required:
   - **EasyDrive_client.py:** Collects customer information such as name, address, PPS number, and driving license document through a console interface.
   - **EasyDrive_server.py:** Receives data from the client, processes it, and stores it in a database.
3. Utilised **TCP (Transmission Control Protocol)** using Python's *socket* library to establish a **connection-oriented communication** between client and server.
4. Designed the server to:
   - Accept multiple client connections using **multithreading**

- Receive and parse customer data
- Generate a **unique registration number** for each user
- Send the registration number back to the client

5. Integrated a **SQLite database *(customers.db)*** for **persistent data storage**, ensuring customer records are saved permanently.
6. Demonstrated use of **data formats and storage** by handling structured data transmission and database insertion.
7. Applied external libraries ***(socket, sqlite3)*** to build a functional network-based system.
8. The task highlights practical implementation of **network programming, database integration, and real-time data exchange**, fulfilling real-world system requirements.

Task 4: Web Scraping and CSV Data Handling (Python)

1. Developed a Python program to scrape book data (name, rating, and price) from a webpage using web scraping techniques.
2. Implemented the solution in a single file:
   - **Scrapper.py**: Handles data extraction, storage, and retrieval.
3. Utilised external libraries:
   - ***requests*** to fetch HTML content from the webpage
   - ***BeautifulSoup*** to parse and extract required data from HTML structure
4. Extracted key information for each book:
   - **Book name** from the title attribute
   - **Rating** from HTML class values
   - **Price** from the price field
5. Stored the extracted data in a **CSV file (books.csv)** using Python's built-in csv module, demonstrating file-based data handling.
6. Implemented functionality to **read data back from the CSV file** and display it in the terminal, ensuring full data flow from extraction to output.
7. Resolved encoding issues by using appropriate **UTF-8 encoding**, ensuring correct display of currency symbols.
8. Demonstrated use of **HTML data format**, file handling, and integration of external libraries to build an efficient data extraction system.

9. This task highlights practical application of **web scraping, data processing, and file storage techniques** in Python.

Conclusion

The given assignment was a real-life experience in applying the concepts of programming in C# and Python. It also showed how to use proper data formats like queues and dictionaries and how to handle various formats of data like databases, CSV, and HTML. Besides, external library and API integration allowed the creation of clients and server and web scraping applications. On the whole, the assignments improved problem-solving skills and reinforced the knowledge of the real-world system design and implementation.

References

Python Software Foundation (2024) *Python Documentation*. Available at: https://docs.python.org/3/.

Microsoft (2024) *C# Documentation*. Available at: https://learn.microsoft.com/en-us/dotnet/csharp/.

Leonard Richardson (2007) *Beautiful Soup Documentation*. Available at: https://www.crummy.com/software/BeautifulSoup/.

SQLite Consortium (2024) *SQLite Documentation*. Available at: https://www.sqlite.org/docs.html.

Books to Scrape (2024) *Travel Books Page*. Available at: https://books.toscrape.com/catalogue/category/books/travel_2/index.html.

Requests Library (2024) *Requests Documentation*. Available at: https://docs.python-requests.org/.